



Universidad Simón Bolívar

Decanato de Estudios Profesionales

Departamento de Computación y Tecnología de la Información

CI2692 Laboratorio de Algoritmos y Estructuras de Datos II

Trimestre Abril-Julio 2026

Profesores: Leonardo López - Kevin Oporto

Septiembre 2026

Laboratorio 6: TAD, Diccionario, Tablas de Hash y Árboles de Decisión

1. TAD Diccionario y Tablas de Hash

1.1. Introducción

El objetivo de este laboratorio es presentar una especificación del TAD Diccionario y realizar dos implementaciones concretas basadas en tablas de hash. La primera implementación consiste en una tabla de hash basada en encadenamiento. La segunda implementación es una tabla de hash basada en direccionamiento abierto, en específico, la *cuckoo hashing*. Para garantizar la consistencia entre la especificación del TAD Diccionario y las implementaciones, para cada una de las implementaciones se va a definir la relación de acoplamiento entre el TAD Diccionario y las tablas de hash.

1.2. TAD Diccionario

La especificación formal del TAD Diccionario se basa en la presentada en [3] y en la clase de teoría de Algoritmos y Estructuras II:

- **Modelo de Representación:**

- `var conocidas : set of T0`
- `var tabla : T0  $\rightarrow$  T1`

- **Invariante de Representación:**

- `conocidas = dom(tabla)`

- **Operaciones:**

- `proc crear (out d : Diccionario)`
 `{Pre: true}`
 `{Post: d.conocidas =  $\emptyset$   $\wedge$  d.tabla =  $\emptyset$ }`
- `proc agregar (in-out d : Diccionario, in clave : T0, in valor : T1)`
 `{Pre: clave  $\notin$  d.conocidas}`
 `{Post: d.conocidas = d0.conocidas  $\cup$  {clave}  $\wedge$  d.tabla = d0.tabla  $\cup$  {(clave, valor)}}`

- `proc eliminar (in-out d : Diccionario, in clave : T0)`
 $\{Pre: clave \in d.conocidas\}$
 $\{Post: d.conocidas = d_0.conocidas - \{clave\} \wedge d.tabla = d_0.tabla - \{(clave, d_0.tabla(clave))\}\}$
- `proc buscar (in d : Diccionario, in clave : T0, out valor : T1)`
 $\{Pre: clave \in d.conocidas\}$
 $\{Post: valor = d.tabla(clave)\}$
- `proc existe (in d : Diccionario, in clave : T0, out r : Boolean)`
 $\{Pre: true\}$
 $\{Post: r \equiv (clave \in d.conocidas)\}$
- `proc toString (in d : Diccionario, out s : String)`
 $\{Pre: true\}$
 $\{Post: s = \text{String que es una representación de los pares } (clave, valor) \text{ contenidos en } d.tabla\}$
- `proc numElementos (in d : Diccionario, out n : Int)`
 $\{Pre: true\}$
 $\{Post: n = \#conocidas\}$

1.3. Tabla de Hash Basada en Encadenamiento

Se quiere que realice una primera implementación del TAD Diccionario por medio de la estructura de datos tabla de hash, usando el método de encadenamiento para la resolución de colisiones. Usted debe implementar una tabla de hash en donde las claves son de tipo entero, y el valor a almacenar que está asociado a cada clave, es un elemento de tipo `String`. El encadenamiento debe ser llevado a cabo mediante una lista doblemente enlazada.

Como en la tabla de hash tenemos claves asociadas a valores, se establece la siguiente relación de acoplamiento entre la tabla de hash basada en encadenamiento y el TAD Diccionario:

$conocidas = \{claves \mid \text{Las claves corresponden a las claves en la tabla de hash}\}$

$\wedge$

$tabla = \{(clave, valor) \mid \text{Pares de claves y valores asociados en la tabla de hash}\}$

Debe implementar una tabla de hash dinámica, es decir, el tamaño de la tabla crece si la tabla tiene un factor de carga mayor o igual a un límite establecido. Para este laboratorio, el tamaño inicial de la tabla es de siete casillas. Si el factor de carga llega a ser igual o mayor a 0.7 durante las operaciones de la tabla, entonces el tamaño n de la tabla se incrementa aplicando la función:

$$incr(n) = \text{int}((n + 16) \times 3/2)$$

donde `int` es una función que redondea un número real a un número entero. Esta operación se conoce como *rehashing*.

La lista doblemente enlazada estará constituida por elementos de tipo `HashTableEntry`. El tipo `HashTableEntry` debe contener al menos cuatro campos: uno para la clave, uno para el valor tipo `String` y dos para las referencias a otros dos elementos de tipo `HashTableEntry`. El tipo `HashTableEntry` debe ser implementado como una clase de Kotlin, cuyo constructor recibe una clave de tipo entero y un valor de tipo `String`.

La función de hash que debe ser usada es el método de la división. De esta implementación son al menos tres los archivos que debe entregar:

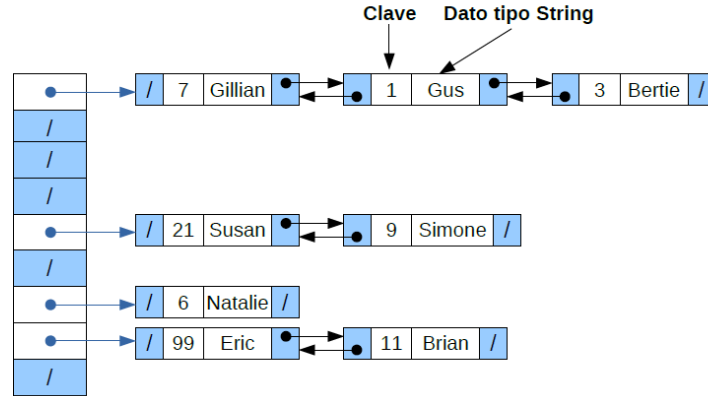


Figura 1: Estructura de la Tabla de Hash basada en encadenamiento con lista doblemente enlazada.

- **HashTableEntry.kt:** Contiene una clase con la implementación del tipo de datos `HashTableEntry`.
- **CircularList.kt:** En este archivo debe estar implementada la lista doblemente enlazada con la clase `CircularList`. La lista debe contener elementos de tipo `HashTableEntry`.
- **HashTableChaining.kt:** En este archivo se implementa el tipo de datos tabla de hash basada en encadenamiento, en la clase `HashTableChaining`. La clase debe hacer uso de los tipos de datos `HashTableEntry` y `CircularList`.

1.4. Cuco Hashing

La segunda implementación concreta del TAD Diccionario está basada en la estructura de datos tabla de hash que usa el método de *cuco hashing* [2] para la resolución de colisiones. Debe implementar una tabla que usa *cuco hashing* en donde las claves son de tipo entero, y el valor que está asociado a cada clave es un elemento de tipo `String`.

Como en el *cuco hashing* tenemos claves asociadas a valores, entonces se puede establecer la siguiente relación de acoplamiento con el TAD Diccionario:

$$\text{conocidas} = \{\text{claves} \mid \text{Las claves corresponden a las claves en la cuco hashing}\}$$

$$\wedge$$

$$\text{tabla} = \{(\text{clave}, \text{valor}) \mid \text{Pares de claves y valores asociados en la cuco hashing}\}$$

La *cuco hashing* es dinámica. En este caso, se debe hacer *rehashing* cuando el factor de carga de la tabla es igual o mayor a 0.7. Se debe usar la misma función indicada para la tabla de hash basada en encadenamiento para el cálculo del tamaño de las nuevas tablas cuando se hace *rehashing*. Cuando se crea una *cuco hashing*, su tamaño inicial es de siete.

Cada par clave-valor en el *cuco hashing* debe estar contenido en una estructura de datos llamada `CuckooHashTableEntry`. El tipo `CuckooHashTableEntry` debe poseer dos campos: el primero se llama `clave`, de tipo entero, y el segundo se llama `valor`, de tipo `String`. El tipo `CuckooHashTableEntry` debe ser implementado como una clase de Kotlin en el archivo `CuckooHashTableEntry.kt`. El constructor de la clase recibe como entrada una clave de tipo entero y un valor asociado de tipo `String`.

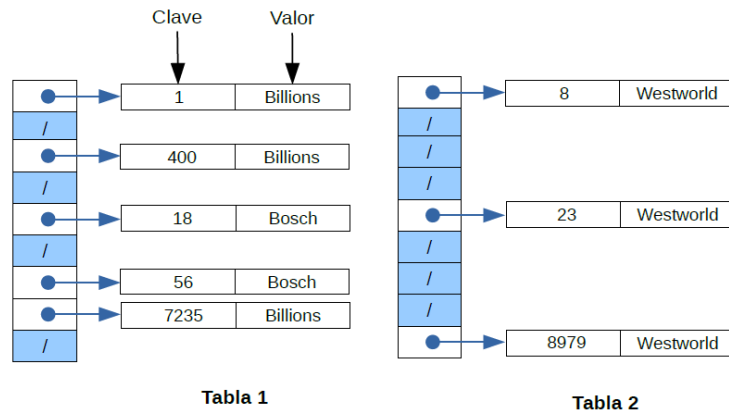


Figura 2: Esquema de funcionamiento y colisiones en *Cuco Hashing*.

La tabla de hash usando *cuco hashing* debe ser implementada como una clase de Kotlin llamada `CuckooHashTable`. El desarrollo de la tabla debe tener como base el pseudocódigo presentado en la clase del curso de teoría. La tabla almacenará elementos de tipo `CuckooHashTableEntry` y debe poseer dos arreglos que contienen a los elementos de este tipo.

El método de *cuco hashing* hace uso de dos funciones de hash h_1 y h_2 :

- La función h_1 corresponde al método de la división.
- Para la función h_2 se debe usar el método de la multiplicación, usando como constante A el valor sugerido por Knuth [1] de $A = 0,6180339887$.

Su implementación de la *cuco hashing* debe contener, al menos, los siguientes archivos:

- `CuckooHashTableEntry.kt`: Implementación de la clase `CuckooHashTableEntry`.
- `CuckooHashTable.kt`: Implementación de la *cuco hashing* en la clase `CuckooHashTable`.

1.4.1. Programa de Pruebas de las Tablas de Hash

Se desea hacer una comparación experimental del rendimiento de la tabla de hash basada en *cuco hashing* y la tabla de hash basada en encadenamiento. El objetivo es comparar el rendimiento de ambas tablas bajo un conjunto de operaciones y usando diferente cantidad de datos.

Debe realizar un programa cliente llamado `Main.kt`, el cual compara las dos tablas de hash mediante la siguiente prueba:

1. Se crea un arreglo, el cual va a almacenar n números enteros generados aleatoriamente que se encuentran en el intervalo $[0, \frac{n}{3}]$. Este arreglo contiene las claves que van a ser agregadas en las tablas. Es decir, las dos tablas de hash deben ser probadas sobre la misma secuencia de claves.
2. El valor asociado a cada clave se obtiene convirtiendo en `String` cada una de las claves. De esa manera se debe crear un arreglo de pares (`clave`, `valor`) de elementos a insertar en la tabla de hash.
3. Para cada uno de los elementos del arreglo de pares (`clave`, `valor`), se busca si el elemento existe en la tabla de hash; si existe se elimina, de lo contrario se agrega.

4. Se debe medir el tiempo usado por la tabla de hash para procesar el arreglo de pares (`clave`, `valor`) (los pasos 2 y 3 indicados anteriormente). No se debe tomar en cuenta el tiempo usado para la creación de este arreglo.

El procedimiento descrito debe ser aplicado a las dos tablas de hash a comparar. El cliente `Main.kt` se debe ejecutar desde un script llamado `runTestHashTables.sh`, el cual se ejecuta con la siguiente línea de comando:

```
./runTestHashTables.sh <n>
```

Donde n es el número de elementos que contiene el arreglo de pares (`clave`, `valor`) que va a ser procesado. Como resultado, el programa debe mostrar por la salida estándar el tiempo de ejecución usado por cada tabla.

La entrega debe incluir un archivo `Makefile` que compile todos los archivos Kotlin.

1.4.2. Evaluación Experimental y Reporte

Se debe realizar un estudio sobre el rendimiento de las tablas de hash. Se debe ejecutar el programa `runTestHashTables.sh` 5 veces con un valor de $n = 5,000,000$ y se debe obtener el tiempo promedio y la desviación estándar.

Debe realizar un breve reporte con los resultados obtenidos y su análisis en formato PDF, el cual debe contener:

1. La identificación de los estudiantes autores del trabajo.
2. Los datos de la plataforma donde se ejecutaron los algoritmos: sistema operativo, modelo de CPU, cantidad de memoria RAM, versión del compilador Kotlin y versión de la JVM utilizada.
3. Una tabla con el tiempo promedio de cada tabla de hash y su desviación estándar.
4. Un gráfico del tiempo promedio indicado en la tabla anterior, donde se pueda observar la magnitud de la diferencia en los tiempos.
5. Un análisis de los resultados obtenidos.

1.5. Condiciones de entrega

La entrega debe contener los archivos del código de la sección 1 del laboratorio en una carpeta llamada `seccion_1`.

2. Árboles de Decisión

2.1. Introducción

Un Árbol de Decisión es una herramienta que modela una secuencia de decisiones como un árbol. Dado un estado determinado, es posible tomar una decisión entre varias alternativas. Los estados o consecuencias de las decisiones son representados como nodos, mientras que las ramas representan las decisiones que se pueden tomar en cada estado. La Figura 4 muestra un ejemplo de un Árbol de Decisión para la selección de un empleado.

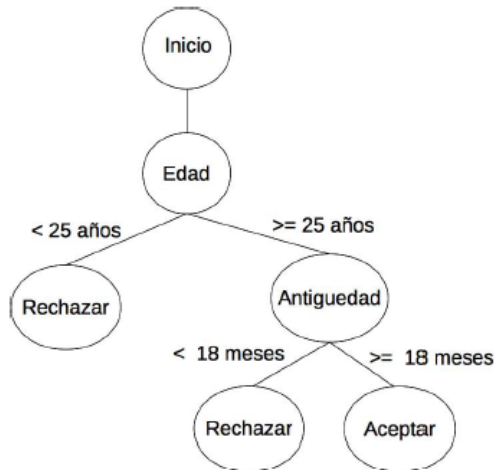


Figura 3: Gráfica comparativa de rendimiento y tiempos de ejecución entre las tablas de hash.

2.2. Código Morse

El Código Morse es un medio para enviar información de texto por medio de una serie de señales de diferente duración y transmisión intermitente. Consiste en que cada letra es transmitida de forma individual con un código que contiene puntos y rayas.

A	.-	F	..-.	K	-. -	P	.-.-.	U	..-
B	-...	G	--.	L	.-..	Q	--.-	V	...-
C	-. -.	H		M	--	R	.-.	W	.-.-
D	-. .	I	..	N	-.	S	...	X	-. -.
E	.	J	.-.-.	O	---	T	-	Y	-. -.
								Z	--..

Figura 4: Árbol de Decisión para la admisión o rechazo de un empleado.

Para decodificar una secuencia de letras o palabras codificadas en Código Morse, se puede modelar el Código Morse como un árbol de decisión como se muestra en la Figura 5. Los nodos del árbol representan las letras del Código Morse, mientras los lados indican los puntos y rayas que hay que escoger para la composición de las letras. La raíz del árbol es el carácter vacío y es así porque desde la raíz comienza la decodificación de una secuencia de puntos y rayas.

El procedimiento para decodificar una letra es como sigue: se comienza a recorrer el árbol desde la raíz y se sigue la rama derecha o izquierda, dependiendo del símbolo actual de la secuencia que se esté examinando. Si el símbolo es un punto, se recorre el árbol por el hijo izquierdo, y si es una raya por el hijo derecho. Se sigue así para todos los símbolos. Si al final del recorrido se llega a una letra, entonces la misma corresponde al código; si por el contrario es NIL, entonces el código está errado.

Por ejemplo, para decodificar el símbolo `-. .` comenzamos desde la raíz. Luego examinamos el primer símbolo y es una raya, por lo tanto seguimos por la rama derecha y alcanzamos el nodo **"T"**. Examinamos el siguiente símbolo y es otra raya, por lo tanto seleccionamos la rama derecha y llegamos al nodo con la letra **"M"**. Revisamos el último símbolo de la secuencia y es un punto, por lo que tenemos que tomar la rama izquierda, la cual nos hace llegar al nodo con la letra **"G"**. Debido a que este es el último nodo al cual arribamos, la secuencia corresponde a la letra **"G"**. En caso de que se intente decodificar una secuencia que es inválida, vamos a terminar el recorrido del árbol en una hoja NIL.

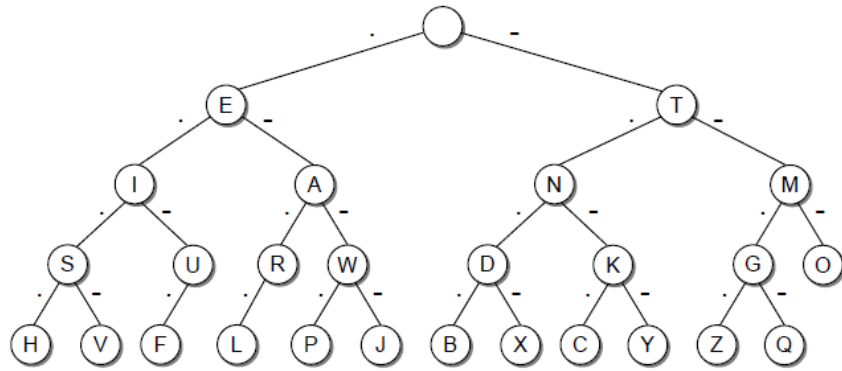


Figura 5: Código Morse modelado como Árbol de Decisión binario.

2.3. Actividad a realizar

Se quiere que implemente el TAD Código Morse usando la programación orientada a objetos. El TAD Código Morse posee, al menos, los siguientes métodos:

- **Crear Código Morse:** Constructor de la clase. Crea un árbol de decisión con las letras del alfabeto latino básico (ISO).

```
constructor CrearCodigoMorse()
```

- **Decodificar Letra:** Función que decodifica una secuencia en Código Morse para obtener la letra que corresponde. Si la secuencia es válida, se retorna la letra correspondiente, en otro caso se retorna Null.

```
func decodificarLetra (secuencia: String) -> String
```

- **Decodificar Mensaje:** Función que decodifica una frase escrita con letras en Código Morse. Si todas las letras en Código Morse son válidas, retorna un **String** con la traducción de la frase. En caso contrario retorna Null. Como convención para separar las letras de una palabra se usará un espacio en blanco y para separar palabras se usará una barra /. Por ejemplo la frase:

```
.-... --- .../.- .-. -... --- .-. . .../... --- -./-.- ..... -.- .-. ....
```

se traduce como "los arboles son cheveres".

```
func decodificarMensaje (frase: String) -> String
```

2.4. Detalles de la implementación

Debe implementar el TAD Código Morse como una clase pública de Kotlin, llamada **CodigoMorseAD**. Esta clase debe estar contenida en un archivo llamado **CodigoMorseAD.kt**.

Se quiere que realice un programa cliente para probar el TAD Código Morse. Este cliente se llama **PruebaCodigoMorseAD.kt**, y se ejecuta por la línea de comandos, recibiendo como entrada un **String** en código Morse, y muestra por la salida estándar el código de entrada decodificado. Si el código Morse no es válido, entonces el cliente muestra la frase **'Error, codigo Morse no valido'**. Para ejecutar el cliente debe realizar un shell script, llamado **runCodigoMorse.sh**, que recibe como entrada el **String** a decodificar. Por último, debe entregar un archivo **Makefile** que compile todos los archivos del proyecto.

2.5. Condiciones de entrega

La entrega debe contener los archivos del código de la sección 2 del laboratorio en una carpeta llamada `seccion_2`.

3. Condiciones de entrega final

La entrega final debe contener las 2 carpetas `seccion_1` y `seccion_2`, las cuales deben estar contenidas en un archivo comprimido, con formato `tar.xz`, llamado `LabSem11_X.Y.tar.xz`, donde X e Y son los números de carné de los estudiantes. La entrega debe hacerse por la plataforma Classroom, antes de las 11:59 PM del día jueves 17 de septiembre de 2026.

Referencias

- [1] CORMEN, T., LEISERSON, C., RIVEST, R., AND STEIN, C. *Introduction to algorithms*, 3rd ed. MIT press, 2009.
- [2] PAGH, R., AND RODLER, F. F. *Cuckoo hashing*. Journal of Algorithms 51, 2 (2004), 122-144.
- [3] RAVELO, J. *Especificación e implementación de tipos abstractos de datos*. <http://ldc.usb.ve/~jravelo/docencia/algoritmos/material/tads.pdf>, 2012.

Leonardo López / leonardolopez@usb.ve
Kevin Oporto / kevinoporto.h@gmail.com
Septiembre 2026